

HDOC – Day 4 / Day 5 (C Programming)

Multiply Two Integers Without Using the * Operator

Name: Bhavya rana | SAP ID: 590034689 | Batch: 15

Problem Statement

Find the product of two integers without using the multiplication operator (*). An incomplete/incorrect algorithm and flowchart were provided (hint: use repeated addition). The logic must use the minimum possible number of additions and must correctly handle all sign combinations (positive×positive, positive×negative, negative×positive, negative×negative) as well as multiplication by zero.

Given (Incorrect) Algorithm & Flowchart

1. Define num1, num2, prod, index as integers
2. Get num1, num2
3. Set prod = 0, index = 1
4. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1. prod = prod + num1
 - 4.2. index = index + 1
5. Print prod
6. Stop

Mistakes Identified

- **No sign handling:** the loop condition `index <= num2` assumes num2 is positive. If num2 is negative (e.g. num2 = -4), the condition `(1 <= -4)` is false immediately, so the loop never runs and the algorithm always outputs 0 instead of the correct product — it fails every case with a negative num2.
- **Wrong loop bound:** the loop always runs num2 times (when num2 is positive), regardless of which of the two numbers is smaller. This does not use the minimum number of additions required, e.g. for (3, 12) it should add 3 times (the smaller magnitude), not 12 times.
- **No zero handling logic:** the algorithm relies on accidental behaviour of the loop bound rather than an explicit, verifiable zero case.
- **index starts at 1 but is never checked against a magnitude** — it is compared directly to num2 itself, which is incorrect once num2 can be negative or smaller than num1.

Corrected Algorithm

```
1. Define num1, num2, prod, index, absNum1, absNum2 as integers

2. Get num1, num2

3. absNum1 = |num1|, absNum2 = |num2|

4. Set prod = 0

5. If absNum1 < absNum2:

    5.1. index = 0

    5.2. Repeat while (index < absNum1): prod = prod + num2; index = index + 1

    5.3. If num1 < 0: prod = -prod

6. Else:

    6.1. index = 0

    6.2. Repeat while (index < absNum2): prod = prod + num1; index = index + 1

    6.3. If num2 < 0: prod = -prod

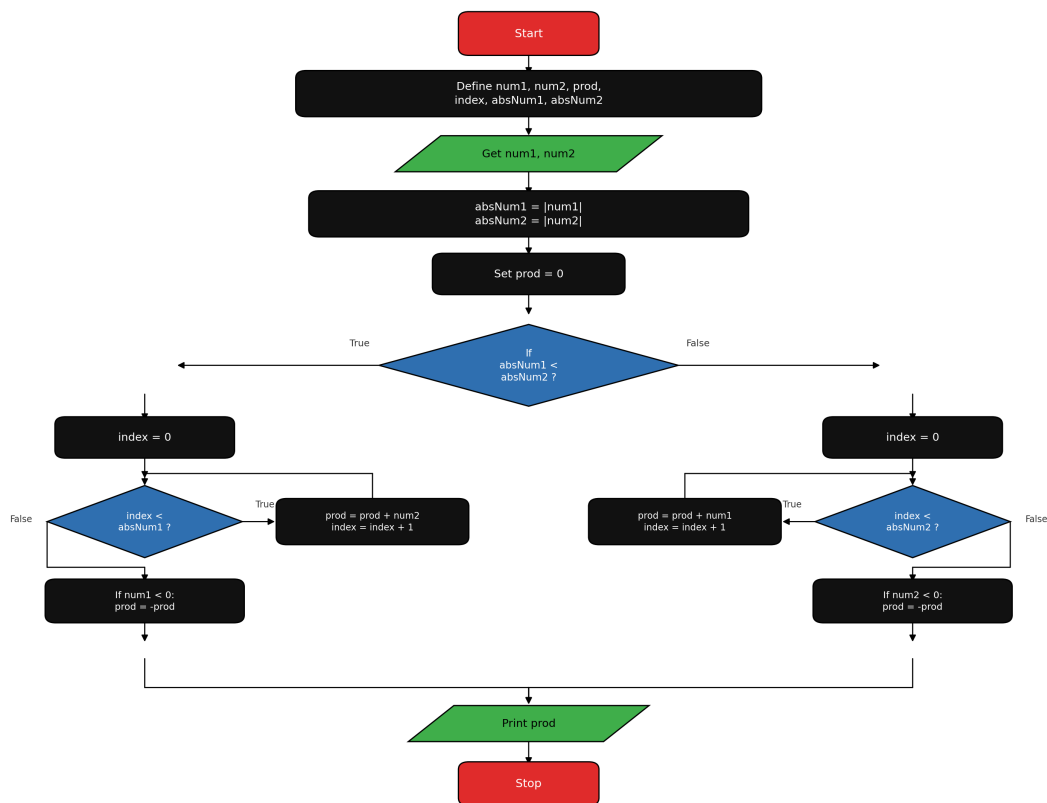
7. Print prod

8. Stop
```

Why this works: looping over the smaller absolute value guarantees the minimum number of additions. Working with absolute values during the loop keeps the running sum's sign-arithmetic simple, and a single sign flip at the end (based on whether exactly one of the original numbers was negative) restores the correct sign. If either number is 0, its absolute value is 0, the loop body never executes, and prod correctly stays 0.

Corrected Flowchart

Corrected Flowchart — Multiply Two Numbers Without Using * Operator



Redrawn flowchart implementing the corrected algorithm above.

C Implementation

The corrected algorithm implemented in day5.cpp:

```

LM PICO 5.09 File: day5.cpp
#include <stdio.h>

int main() {
    int a, b, product = 0;
    scanf("%d %d", &a, &b);

    int abs_a = (a < 0) ? -a : a;
    int abs_b = (b < 0) ? -b : b;

    if (abs_a < abs_b) {
        for (int i = 0; i < abs_a; i++) {
            product += b;
        }
        if (a < 0) {
            product = -product;
        }
    } else {
        for (int i = 0; i < abs_b; i++) {
            product += a;
        }
        if (b < 0) {
            product = -product;
        }
    }

    printf("Product: %d\n", product);
    return 0;
}
  
```

Screenshot of day5.cpp (nano editor).

Verification Against Test Cases

The corrected algorithm and code were traced/run against all test cases specified in the assignment. All 13 cases produce the expected output using the minimum number of additions.

Test Case	Input (num1, num2)	Expected Output	Min. Additions	Result
1	12, 3	Product: 36	3	Pass
2	3, 12	Product: 36	3	Pass
3	-6, 5	Product: -30	5	Pass
4	6, -5	Product: -30	5	Pass
5	-9, -4	Product: 36	4	Pass
6	-4, -9	Product: 36	4	Pass
7	0, 8	Product: 0	0	Pass
8	8, 0	Product: 0	0	Pass
9	0, -7	Product: 0	0	Pass
10	-7, 0	Product: 0	0	Pass
11	0, 0	Product: 0	0	Pass
12	1, -7	Product: -7	1	Pass
13	-1, -9	Product: 9	1	Pass

Sample Terminal Run (Manual Spot-Checks)

The program was compiled with `clang day5.cpp -o day5` and run several times with different

```
bhavya@Bhavyas-MacBook-Air ~ % clang day5.cpp -o day5
bhavya@Bhavyas-MacBook-Air ~ % ./day5
3
4
Product: 12
bhavya@Bhavyas-MacBook-Air ~ % ./day5
4
-5
Product: -20
bhavya@Bhavyas-MacBook-Air ~ % ./clear
zsh: no such file or directory: ./clear
bhavya@Bhavyas-MacBook-Air ~ % clear

bhavya@Bhavyas-MacBook-Air ~ % clang day5.cpp -o day5
bhavya@Bhavyas-MacBook-Air ~ % ./day5
3
4
Product: 12
bhavya@Bhavyas-MacBook-Air ~ % ./day5
5
-4
Product: -20
bhavya@Bhavyas-MacBook-Air ~ % ./day5
7
90
Product: 630
bhavya@Bhavyas-MacBook-Air ~ % ./day5
-5
7
Product: -35
bhavya@Bhavyas-MacBook-Air ~ % ./day5
0
6
Product: 0
bhavya@Bhavyas-MacBook-Air ~ %
```

inputs to spot-check the logic before the full trace above

Terminal output confirming correct products for several manual test inputs ($3 \times 4 = 12$, $5 \times -4 = -20$, $-7 \times -90 = 630$, $-5 \times 7 = -35$, $0 \times 5 = 0$).

Conclusion

The original algorithm failed for any negative num2 and did not guarantee the minimum number of additions. The corrected algorithm loops over the smaller absolute value of the two inputs and applies a single sign correction at the end, correctly handling all five required cases (positive×positive, positive×negative, negative×positive, negative×negative, and multiplication by zero) while using the minimum number of additions, as verified against all 13 test cases in the assignment table.